

S.Aishwarya
2303A51833

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																																					
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026																																				
Course Coordinator Name		Dr. Rishabh Mittal																																					
Instructor(s)Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td><td></td></tr> <tr><td>Ms. B. Swathi</td><td></td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td><td></td></tr> <tr><td>Mr. Md Sallauddin</td><td></td></tr> <tr><td>Dr. Mathivanan</td><td></td></tr> <tr><td>Mr. Y Srikanth</td><td></td></tr> <tr><td>Ms. N Shilpa</td><td></td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td><td></td></tr> <tr><td>Dr. R. Prashant Kumar</td><td></td></tr> <tr><td>Mr. Ankushavali MD</td><td></td></tr> <tr><td>Mr. B Viswanath</td><td></td></tr> <tr><td>Ms. Sujitha Reddy</td><td></td></tr> <tr><td>Ms. A. Anitha</td><td></td></tr> <tr><td>Ms. M.Madhuri</td><td></td></tr> <tr><td>Ms. Katherashala Swetha</td><td></td></tr> <tr><td>Ms. Velpula sumalatha</td><td></td></tr> <tr><td>Mr. Bingi Raju</td><td></td></tr> <tr><td>Mr. G. Kranthi</td><td></td></tr> </table>		Mr. S Naresh Kumar		Ms. B. Swathi		Dr. Sasanko Shekhar Gantayat		Mr. Md Sallauddin		Dr. Mathivanan		Mr. Y Srikanth		Ms. N Shilpa		Dr. Rishabh Mittal (Coordinator)		Dr. R. Prashant Kumar		Mr. Ankushavali MD		Mr. B Viswanath		Ms. Sujitha Reddy		Ms. A. Anitha		Ms. M.Madhuri		Ms. Katherashala Swetha		Ms. Velpula sumalatha		Mr. Bingi Raju		Mr. G. Kranthi	
Mr. S Naresh Kumar																																							
Ms. B. Swathi																																							
Dr. Sasanko Shekhar Gantayat																																							
Mr. Md Sallauddin																																							
Dr. Mathivanan																																							
Mr. Y Srikanth																																							
Ms. N Shilpa																																							
Dr. Rishabh Mittal (Coordinator)																																							
Dr. R. Prashant Kumar																																							
Mr. Ankushavali MD																																							
Mr. B Viswanath																																							
Ms. Sujitha Reddy																																							
Ms. A. Anitha																																							
Ms. M.Madhuri																																							
Ms. Katherashala Swetha																																							
Ms. Velpula sumalatha																																							
Mr. Bingi Raju																																							
Mr. G. Kranthi																																							
Course Code	23CS002PC304	Course Title	AI Assisted Coding																																				
Year/Sem	III/I	Regulation	R23																																				
Date and Day of Assignment	Week 7 - Thursday	Time(s)	23CSBTB01 To 23CSBTB52																																				
Duration	2 Hours	Applicable to Batches	All Batches																																				
AssignmentNumber: 14.4 (Present assignment number)/24(Total number of assignments)																																							
Q.No.	Question	ExpectedTime to complete																																					
1	Lab 14: Web Frontend Development – AI-Assisted HTML/CSS/JS Generation	Week 7																																					

	Lab Objectives • Design functional, visually appealing web applications using HTML, CSS, and JavaScript with AI assistance. • Apply responsive, accessible, and interactive design principles. • Create practical UI components for real-world web applications. • Use AI to optimize layout, UX, and performance. Lab Outcomes • Applying AI assistance to frontend development tasks • Understanding how HTML, CSS, and JavaScript work together • Improving responsiveness, interactivity, and usability • Evaluating and refining AI-generated frontend code	
	Task 1: Responsive Homepage Layout for a Startup Website Scenario You are designing a basic homepage for a startup company that needs to look good on desktops, tablets, and mobile devices. Task Description • Create a web page structure using HTML that includes: ◦ A header with navigation links ◦ A main content section ◦ A footer with basic information • Use AI assistance to generate responsive CSS using media queries. • Ensure the layout adjusts smoothly across different screen sizes. Expected Output • A responsive web page where: ◦ Header, content, and footer are clearly separated ◦ Layout adapts correctly to mobile, tablet, and desktop views ◦ Code is clean and well-structured Prompt:-	

create a web page structure using Html which includes a header with navigation links with a main content section with a footer with basic information. also add css using media queries and ensure smooth layout handing

Code:-

lab 14.4 > index.html > html

```
2 <html lang="en">
3 <head>
7 <style>
39   nav ul {
40     list-style: none;
41     display: flex;
42     justify-content: center;
43     flex-wrap: wrap;
44   }
45
46   nav ul li {
47     margin: 0;
48   }
49
50   nav ul li a {
51     display: block;
52     color: white;
53     text-decoration: none;
54     padding: 1rem 1.5rem;
55     transition: background-color 0.3s ease;
56   }
57
58   nav ul li a:hover {
59     background-color: #2980b9;
60   }
61
62   main {
63     max-width: 1200px;
64     margin: 2rem auto;
65     padding: 0 1rem;
66     min-height: calc(100vh - 300px);
67   }
68
69   .content-section {
70     background-color: #ecf0f1;
71     padding: 2rem;
72     margin-bottom: 2rem;
```

lab 14.4 > index.html > html

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Responsive Web Page</title>
7   <style>
8     * {
9       margin: 0;
10      padding: 0;
11      box-sizing: border-box;
12    }
13
14    body {
15      font-family: Arial, sans-serif;
16      line-height: 1.6;
17      color: #333;
18    }
19
20    header {
21      background-color: #2c3e50;
22      color: white;
23      padding: 1rem 0;
24      position: sticky;
25      top: 0;
26      z-index: 100;
27    }
28
29    header h1 {
30      text-align: center;
31      margin-bottom: 0.5rem;
32    }
33
34    nav {
35      background-color: #34495e;
36      padding: 0;
37    }
```

```

lab 14.4 > index.html > html
2  <html lang="en">
3  <head>
7  <style>
76  .content-section h2 {
77      color: #2c3e50;
78      margin-bottom: 1rem;
79  }
80
81  footer {
82      background-color: #2c3e50;
83      color: white;
84      text-align: center;
85      padding: 2rem;
86      margin-top: 2rem;
87  }
88
89  footer p {
90      margin: 0.5rem 0;
91  }
92
93  /* Media Queries for Responsive Design */
94  @media (max-width: 768px) {
95      header h1 {
96          font-size: 1.5rem;
97      }
98
99      nav ul {
100          flex-direction: column;
101      }
102
103      nav ul li a {
104          padding: 0.8rem 1rem;
105          text-align: center;
106      }
107
108      main {
109          margin: 1rem auto;

```

```

lab 14.4 > index.html > html
2  <html lang="en">
3  <head>
7  <style>
116  @media (max-width: 480px) {
117      header h1 {
118          font-size: 1.2rem;
119          padding: 0 0.5rem;
120      }
121
122      nav ul li a {
123          padding: 0.6rem 0.8rem;
124          font-size: 0.9rem;
125      }
126
127      main {
128          padding: 0 0.5rem;
129      }
130
131      .content-section {
132          padding: 1rem;
133          margin-bottom: 1rem;
134      }
135
136      footer {
137          padding: 1rem;
138          font-size: 0.9rem;
139      }
140  }
141
142  </style>
143  </head>
144  <body>
145      <header>
146          <h1>My Website</h1>
147      </header>
148
149      <nav>

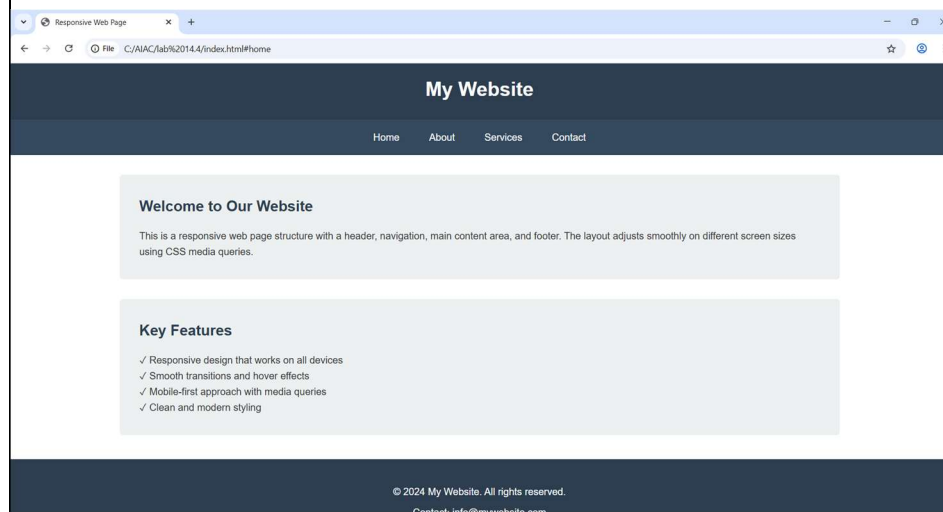
```

```

9.4.py lab exam1.py assign 10.3.py assign 8.3.py assign 10.4.py assign 11.3.py assign 12.4.py index.html X add.py
2 <html lang="en">
144 <body>
145 <header>
147 </header>
148
149 <nav>
150 <ul>
151 <li><a href="#home">Home</a></li>
152 <li><a href="#about">About</a></li>
153 <li><a href="#services">Services</a></li>
154 <li><a href="#contact">Contact</a></li>
155 </ul>
156 </nav>
157
158 <main>
159 <section class="content-section">
160 <h2>Welcome to Our Website</h2>
161 <p>This is a responsive web page structure with a header, navigation, main content area, and footer. The layout adjusts smoothly on
162 </section>
163
164 <section class="content-section">
165 <h2>Key Features</h2>
166 <p>✓ Responsive design that works on all devices</p>
167 <p>✓ Smooth transitions and hover effects</p>
168 <p>✓ Mobile-first approach with media queries</p>
169 <p>✓ Clean and modern styling</p>
170 </section>
171 </main>
172
173 <footer>
174 <p>© 2024 My Website. All rights reserved.</p>
175 <p>Contact: info@mywebsite.com</p>
176 </footer>
177 </body>
178 </html>

```

Output:-



Explanation: -

This task creates a basic webpage layout with a header, main content section, and footer using HTML. Responsive CSS with media queries is used so the layout adjusts properly for desktop, tablet, and mobile screens. This ensures the website looks good on different devices.

Task 2: Interactive Call-to-Action Button

Scenario

A marketing page requires a **call-to-action button** that responds when users interact with it.

Task Description

- Add a button to a web page using HTML.
- Use AI to generate JavaScript that responds to the button click by:
 - Displaying a message or alert
- Ask AI to add meaningful comments explaining the JavaScript logic.

Expected Output

- A web page where:
 - Clicking the button triggers a visible response (alert or message)
 - JavaScript code is readable and commented
 - No page reload occurs

Prompt:-

A marketing page requires a call-to-action button that responds when users interact with it. so add a button to that web page and generate a javascript code that responds to that button click by displaying a message or an alert and also provide meaning ful comments explaining about the javascript

Code:-

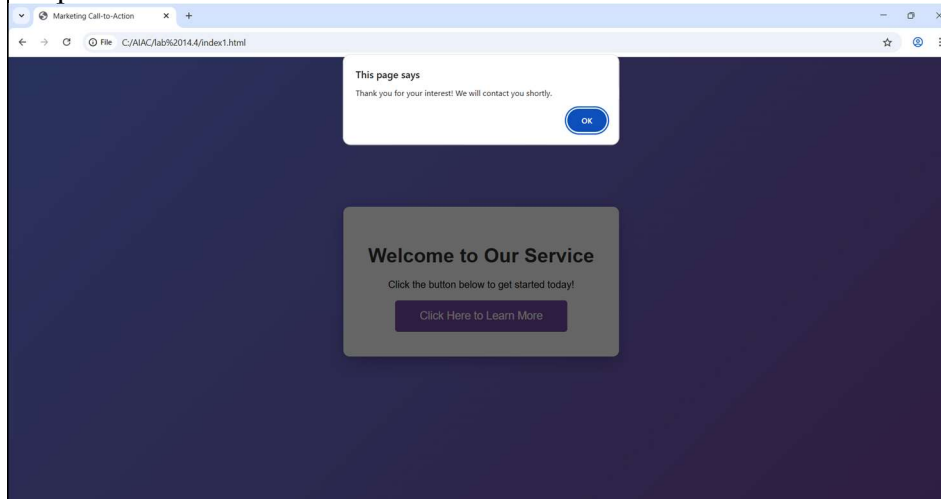
```
lab 14.4 > index1.html > html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Marketing Call-to-Action</title>
7    <style>
8      body {
9        font-family: Arial, sans-serif;
10       display: flex;
11       justify-content: center;
12       align-items: center;
13       height: 100vh;
14       background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
15       margin: 0;
16     }
17     .container {
18       text-align: center;
19       background: #fff;
20       padding: 40px;
21       border-radius: 10px;
22       box-shadow: 0 10px 25px #0000000a;
23     }
24     h1 {
25       color: #333;
26       margin-bottom: 20px;
27     }
28     .cta-button {
29       background-color: #667eea;
30       color: #fff;
31       padding: 15px 40px;
32       font-size: 18px;
33       border: none;
34       border-radius: 5px;
35       cursor: pointer;
36       transition: background-color 0.3s ease;
37     }
```

```

lab 14.4 > index1.html > html
2  <html lang="en">
3  <head>
7    <style>
38      .cta-button:hover {
39        background-color: #764ba2;
40      }
41    </style>
42  </head>
43  <body>
44    <div class="container">
45      <h1>Welcome to Our Service</h1>
46      <p>Click the button below to get started today!</p>
47      <!-- Call-to-action button with id for JavaScript reference -->
48      <button id="ctaBtn" class="cta-button">Click Here to Learn More</button>
49    </div>
50
51    <script>
52      // Get reference to the button element using its id
53      const button = document.getElementById('ctaBtn');
54
55      // Add event listener to the button for 'click' events
56      button.addEventListener('click', function() {
57        // Display an alert message when button is clicked
58        alert('Thank you for your interest! We will contact you shortly.');

```

Output:-



Explanation:-

A call-to-action button is added using HTML, and JavaScript is used to detect when the user clicks the button. When clicked, it displays an alert or message without reloading the page. This makes the webpage interactive and improves user engagement.

Task 3: Contact Form with Client-Side Validation

Scenario

You are developing a **contact form for a company website** to collect user queries safely and correctly.

Task Description

- Design a form with fields for:
 - Name
 - Email
 - Message
- Use AI to generate JavaScript validation logic that:
 - Prevents empty submissions
 - Checks for a valid email format
- Display clear error messages for invalid input.

Expected Output

- A functional form where:
 - Invalid input shows inline error messages
 - Valid input displays a success message
 - Validation happens on the client side using JavaScript

Prompt:-

you should develop a contact form for a company website to collect user queries safely and correctly.Design a form with fields with Name,Email,Message .generate JavaScript validation logic does Prevents empty submissions and Checks for a valid email format , Display clear error messages for invalid input.

Code:-

lab 14.4 > index3.html > html

```
1
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Contact Form</title>
8   <style>
9     * {
10       margin: 0;
11       padding: 0;
12       box-sizing: border-box;
13     }
14
15     body {
16       font-family: Arial, sans-serif;
17       background-color: #f4f4f4;
18       padding: 20px;
19     }
20
21     .container {
22       max-width: 500px;
23       margin: 50px auto;
24       background-color: white;
25       padding: 30px;
26       border-radius: 8px;
27       box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
28     }
29
30     h1 {
31       color: #333;
32       margin-bottom: 30px;
33       text-align: center;
34     }
35
36     .form-group {
37       margin-bottom: 20px;
38     }
39
40     label {
41       display: block;
42       margin-bottom: 8px;
43       color: #333;
44       font-weight: bold;
45     }
```

gn 12.4.py index.html index2.html index3.html X add.py assign-1.4.py assign-3.4.py assign-5.4.py

lab 14.4 > index3.html > html > head > style > body

```
3 <html lang="en">
4 <head>
8 <style>
56   input:focus, textarea:focus {
57     outline: none;
58     border-color: #4CAF50;
59     box-shadow: 0 0 5px rgba(76, 175, 80, 0.3);
60   }
61
62   textarea {
63     resize: vertical;
64     min-height: 120px;
65   }
66
67   .error {
68     color: #d32f2f;
69     font-size: 12px;
70     margin-top: 5px;
71     display: none;
72   }
73
74   .error.show {
75     display: block;
76   }
77
78   input.invalid, textarea.invalid {
79     border-color: #d32f2f;
80     background-color: #ffebee;
81   }
82
83   button {
84     width: 100%;
85     padding: 12px;
86     background-color: #4CAF50;
87     color: white;
88     border: none;
89     border-radius: 4px;
90     font-size: 16px;
91     font-weight: bold;
92     cursor: pointer;
93     transition: background-color 0.3s;
94   }
95
96   button:hover {
97     background-color: #45a049;
```

```

lab 14.4 > index3.html > html > head > style > body
3 <html lang="en">
115 <body>
116   <div class="container">
117     <h1>Contact Us</h1>
118     <form id="contactForm" novalidate>
119       <div class="form-group">
120         <label for="name">Name</label>
121         <input type="text" id="name" name="name" placeholder="Enter your full name">
122         <div class="error" id="nameError"></div>
123       </div>
124       <div class="form-group">
125         <label for="email">Email</label>
126         <input type="email" id="email" name="email" placeholder="Enter your email address">
127         <div class="error" id="emailError"></div>
128       </div>
129       <div class="form-group">
130         <label for="message">Message</label>
131         <textarea id="message" name="message" placeholder="Enter your message"></textarea>
132         <div class="error" id="messageError"></div>
133       </div>
134       <button type="submit">Send Message</button>
135     </form>
136     <div class="success" id="successMessage">✓ Your message has been sent successfully!</div>
137   </div>
138
139   <script>
140     const form = document.getElementById('contactForm');
141     const nameInput = document.getElementById('name');
142     const emailInput = document.getElementById('email');
143     const messageInput = document.getElementById('message');
144     const successMessage = document.getElementById('successMessage');
145
146     // Validation rules
147     const validate = () => {
148       let isValid = true;
149
150       // Validate Name
151       if (!nameInput.value.trim()) {
152         showError('nameError', 'Name is required');
153         nameInput.classList.add('invalid');
154         isValid = false;
155       } else if (nameInput.value.trim().length < 2) {
156

```

gn 12.4.py index.html index2.html index3.html X add.py assign-1.4.py assign-3.4.py assign

```

lab 14.4 > index3.html > html > head > style > body
3 <html lang="en">
115 <body>
142   <script>
150     const validate = () => {
151
167       // Validate Email
168       const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
169       if (!emailInput.value.trim()) {
170         showError('emailError', 'Email is required');
171         emailInput.classList.add('invalid');
172         isValid = false;
173       } else if (!emailRegex.test(emailInput.value.trim())) {
174         showError('emailError', 'Please enter a valid email address');
175         emailInput.classList.add('invalid');
176         isValid = false;
177       } else {
178         clearError('emailError');
179         emailInput.classList.remove('invalid');
180       }
181
182       // Validate Message
183       if (!messageInput.value.trim()) {
184         showError('messageError', 'Message is required');
185         messageInput.classList.add('invalid');
186         isValid = false;
187       } else if (messageInput.value.trim().length < 10) {
188         showError('messageError', 'Message must be at least 10 characters long');
189         messageInput.classList.add('invalid');
190         isValid = false;
191       } else {
192         clearError('messageError');
193         messageInput.classList.remove('invalid');
194       }
195
196       return isValid;
197     };
198
199     // Show error message
200     const showError = (errorId, message) => {
201       const errorElement = document.getElementById(errorId);
202       errorElement.textContent = message;
203       errorElement.classList.add('show');
204     };
205
206     // Clear error message
207     const clearError = (errorId) => {

```

lab 14.4 > index3.html > html > body > script > validate

```
3 <html lang="en">
115 <body>
142 <script>
200     const showError = (errorId, message) => {
204         };
205
206     // Clear error message
207     const clearError = (errorId) => {
208         const errorElement = document.getElementById(errorId);
209         errorElement.textContent = '';
210         errorElement.classList.remove('show');
211     };
212
213     // Clear errors on input
214     nameInput.addEventListener('input', () => clearError('nameError'));
215     emailInput.addEventListener('input', () => clearError('emailError'));
216     messageInput.addEventListener('input', () => clearError('messageError'));
217
218     // Form submission
219     form.addEventListener('submit', (e) => {
220         e.preventDefault();
221         // Click to add a breakpoint
222         successMessage.classList.remove('show');
223
224         if (validate()) {
225             // Simulate sending data
226             console.log({
227                 name: nameInput.value.trim(),
228                 email: emailInput.value.trim(),
229                 message: messageInput.value.trim()
230             });
231
232             // Reset form and show success message
233             form.reset();
234             successMessage.classList.add('show');
235
236             // Hide success message after 5 seconds
237             setTimeout(() => {
238                 successMessage.classList.remove('show');
239             }, 5000);
240         }
241     });
242 </script>
243 </body>
</html>
```

Output:-

Contact Us

Name
Ashwitha

Email
2303A51632@aru.edu.in

Message
good morning to everyone. I am ashwitha

Send Message

The screenshot shows a web browser window with the address bar displaying 'C:/AIAC/lab/20144/index3.html'. The main content is a 'Contact Us' form. The form is titled 'Contact Us' and contains three input fields: 'Name' with the placeholder text 'Enter your full name', 'Email' with the placeholder text 'Enter your email address', and 'Message' with the placeholder text 'Enter your message'. Below these fields is a green button labeled 'Send Message'. At the bottom of the form, there is a green message box that says '✓ Your message has been sent successfully!'.

Explanation:-

A contact form is created with fields for name, email, and message. JavaScript validation checks that all fields are filled and the email format is correct before submission. Error messages are shown for invalid input, and a success message appears for valid entries.

Task 4: Dynamic List Management for a Product Page

Scenario

A product listing page needs to **add or remove items dynamically** without refreshing the page.

Task Description

- Create a simple HTML list to display items.
- Use AI to generate JavaScript that:
 - Adds a new item when an “Add” button is clicked
 - Removes the last item when a “Remove” button is clicked
- Ensure the DOM updates dynamically.

Expected Output

- A web page where:
 - Items can be added and removed dynamically
 - Changes appear instantly without page reload
 - JavaScript logic is clear and maintainable

Prompt:-

A product listing page needs to add or remove items dynamically without refreshing the page. Create a simple HTML list to display items. Use AI to generate JavaScript that adds a new item when an "Add" button is clicked and it should remove the last item when a "Remove" button is clicked. Ensure the DOM updates dynamically.

Code:-

lab 14.4 > > index4.html > html

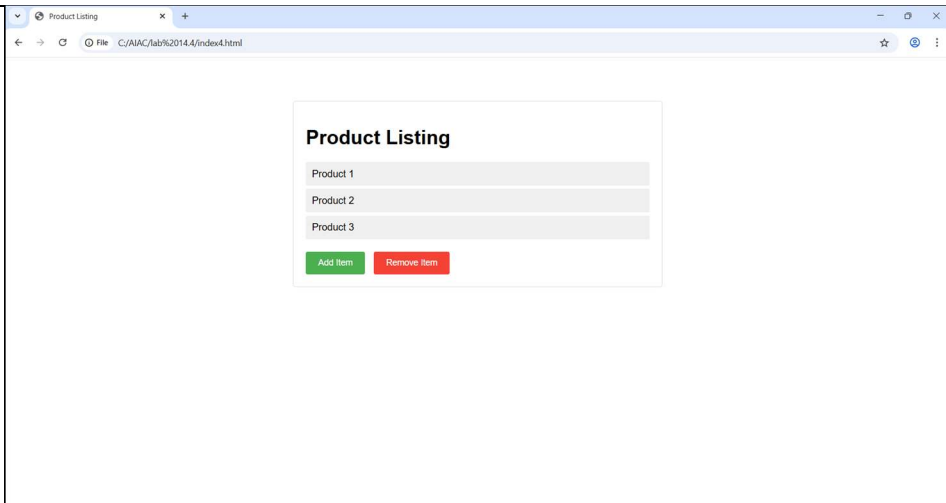
```
4  <!DOCTYPE html>
5  <html lang="en">
6  <head>
7    <meta charset="UTF-8">
8    <meta name="viewport" content="width=device-width, initial-scale=1.0">
9    <title>Product Listing</title>
10   <style>
11     body {
12       font-family: Arial, sans-serif;
13       max-width: 600px;
14       margin: 50px auto;
15       padding: 20px;
16     }
17     .container {
18       border: 1px solid #ddd;
19       padding: 20px;
20       border-radius: 5px;
21     }
22     #productList {
23       list-style-type: none;
24       padding: 0;
25     }
26     #productList li {
27       padding: 10px;
28       margin: 5px 0;
29       background-color: #f0f0f0;
30       border-radius: 3px;
31     }
32     .button-group {
33       margin-top: 20px;
34     }
35     button {
36       padding: 10px 20px;
37       margin-right: 10px;
38       border: none;
39       border-radius: 3px;
40       cursor: pointer;
41       font-size: 14px;
42     }
43     #addBtn {
44       background-color: #4CAF50;
45       color: white;
46     }
```

```

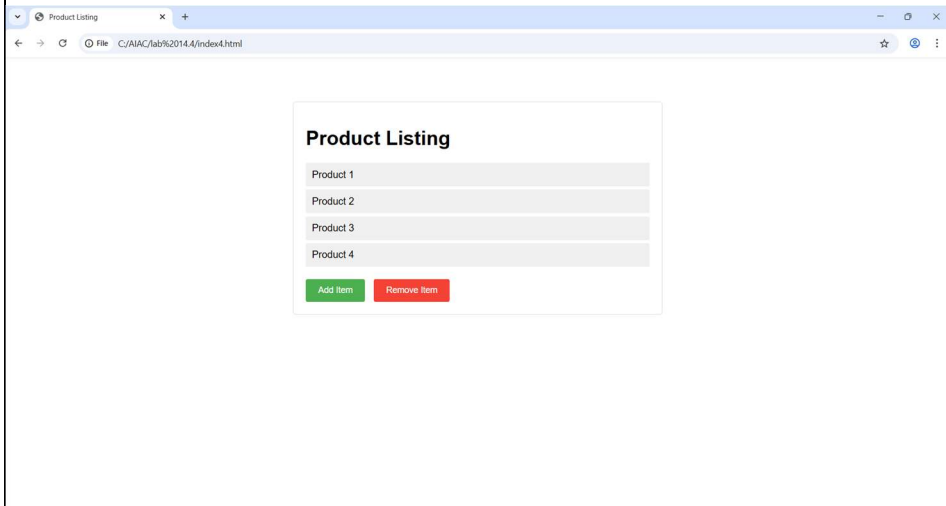
5  <html lang="en">
6  <head>
10  <style>
47  #removeBtn {
48  background-color: #f44336;
49  color: white;
50  }
51  </style>
52  </head>
53  <body>
54  <div class="container">
55  <h1>Product Listing</h1>
56  <ul id="productList">
57  <li>Product 1</li>
58  <li>Product 2</li>
59  <li>Product 3</li>
60  </ul>
61  <div class="button-group">
62  <button id="addBtn">Add Item</button>
63  <button id="removeBtn">Remove Item</button>
64  </div>
65  </div>
66
67  <script>
68  const productList = document.getElementById('productList');
69  const addBtn = document.getElementById('addBtn');
70  const removeBtn = document.getElementById('removeBtn');
71  let itemCount = 3;
72
73  addBtn.addEventListener('click', function() {
74  itemCount++;
75  const newItem = document.createElement('li');
76  newItem.textContent = `Product ${itemCount}`;
77  productList.appendChild(newItem);
78  });
79
80  removeBtn.addEventListener('click', function() {
81  if (productList.children.length > 0) {
82  productList.removeChild(productList.lastChild);
83  }
84  });
85  </script>
86  </body>
87  </html>

```

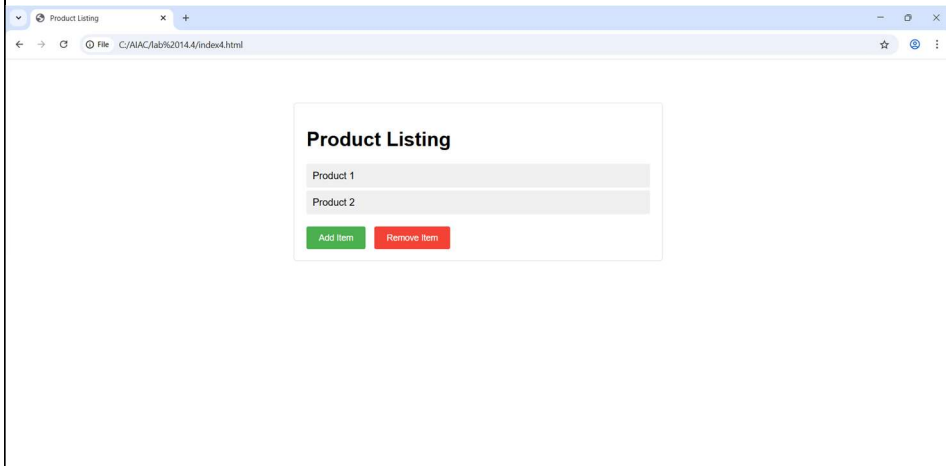
Output:-



Add Item:-



Remove item:-



Explanation:-

An HTML list is used to display items, and JavaScript allows items to be added or removed dynamically. When the “Add” button is clicked, a new item is added to the list, and the “Remove” button deletes the last item. The page updates instantly without reloading.

Task 5: Modal Popup for User Notifications

Scenario

You are building a **user notification popup** for announcements or alerts on a website.

Task Description

- Use AI to help generate:
 - HTML structure for a modal popup
 - CSS styling with a semi-transparent background overlay
 - JavaScript to open and close the modal
- Include multiple ways to close the modal (button or overlay click).

Expected Output

- A web page where:
 - Clicking “Open Modal” displays the popup
 - Popup overlays the page content clearly
 - Modal closes smoothly when instructed

Prompt:-

need to build a user notification popup for announcements or alerts on a website, in that website html structure for a modal pop up and use CSS styling with a semi-transparent background overlay and JavaScript to open and close the modal include multiple ways to close the modal

Code:-

index3.html index4.html Untitled-1 index5.html X add.py assign-1.4.py assign-3.4.py

lab 14.4 > index5.html > html

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Notification Modal Popup</title>
7   <style>
8     * {
9       margin: 0;
10      padding: 0;
11      box-sizing: border-box;
12    }
13
14    body {
15      font-family: Arial, sans-serif;
16      background-color: #f5f5f5;
17      padding: 20px;
18    }
19
20    button {
21      padding: 10px 20px;
22      background-color: #007bff;
23      color: white;
24      border: none;
25      border-radius: 5px;
26      cursor: pointer;
27      font-size: 16px;
28    }
29
30    button:hover {
31      background-color: #0056b3;
32    }
33
34    /* Modal Overlay */
35    .modal-overlay {
36      display: none;
37      position: fixed;
38      top: 0;
39      left: 0;
40      width: 100%;
41      height: 100%;
42      background-color: rgba(0, 0, 0, 0.5);
43      z-index: 999;
44    }
45  </style>
```

```
index3.html  index4.html  Untitled-1  index5.html  X  add.py  assign-1.4.py  assign-3.4.py  a
lab 14.4 > <> index5.html > html
2  <html lang="en">
3  <head>
7  <style>
46  .modal-overlay.active {
47      display: flex;
48      justify-content: center;
49      align-items: center;
50  }
51
52  /* Modal Container */
53  .modal {
54      background-color: white;
55      padding: 30px;
56      border-radius: 10px;
57      box-shadow: 0 4px 6px rgba(0, 0, 0, 0.2);
58      max-width: 500px;
59      width: 90%;
60      position: relative;
61      animation: slideIn 0.3s ease-out;
62  }
63
64  @keyframes slideIn {
65      from {
66          transform: translateY(-50px);
67          opacity: 0;
68      }
69      to {
70          transform: translateY(0);
71          opacity: 1;
72      }
73  }
74
75  /* Close Button (X) */
76  .modal-close-btn {
77      position: absolute;
78      top: 10px;
79      right: 15px;
80      background: none;
81      border: none;
82      font-size: 28px;
83      color: #999;
84      cursor: pointer;
85      padding: 0;
86      width: 30px;
87      height: 30px;
```

lab 14.4 > index5.html > html

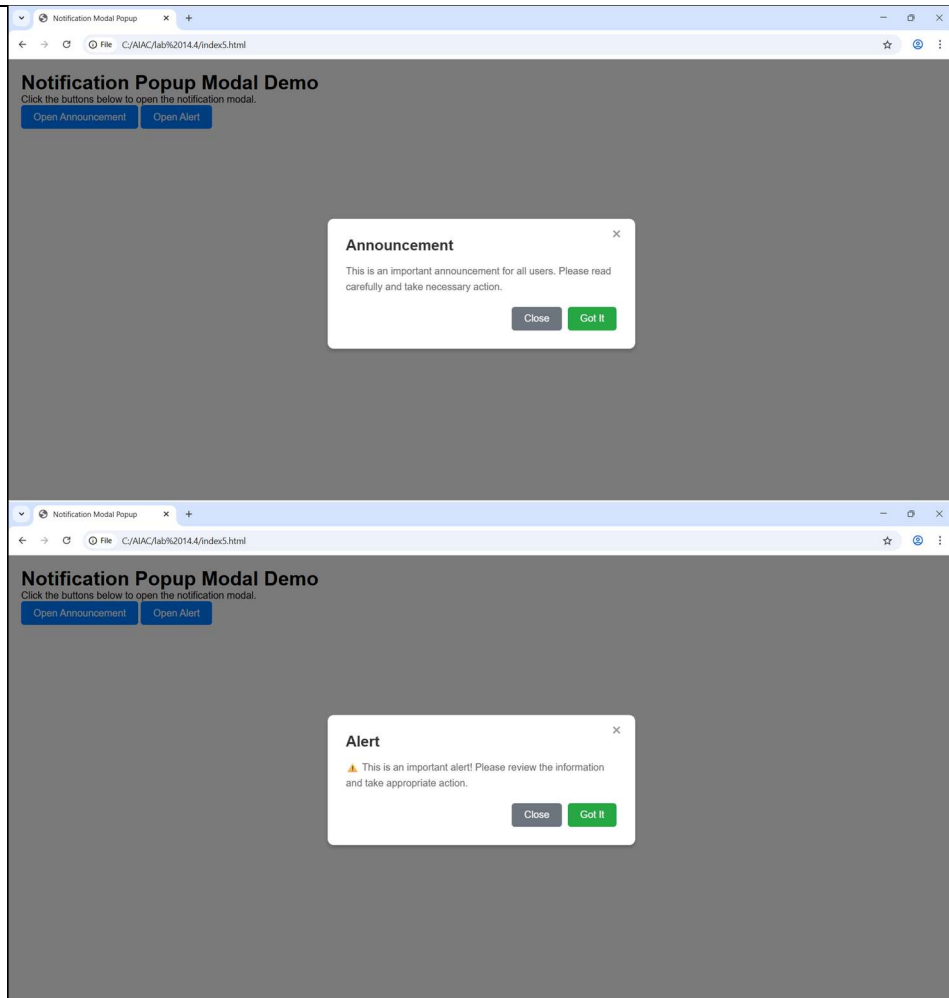
```
2  <html lang="en">
3  <head>
7    <style>
76    .modal-close-btn {
88      display: flex;
89      justify-content: center;
90      align-items: center;
91    }
92
93    .modal-close-btn:hover {
94      color: #333;
95      background-color: #f0f0f0;
96      border-radius: 50%;
97    }
98
99    .modal-header {
100     margin-bottom: 15px;
101     padding-right: 30px;
102   }
103
104   .modal-header h2 {
105     color: #333;
106     margin-bottom: 5px;
107   }
108
109   .modal-body {
110     color: #666;
111     line-height: 1.6;
112     margin-bottom: 20px;
113   }
114
115   .modal-footer {
116     display: flex;
117     gap: 10px;
118     justify-content: flex-end;
119   }
120
121   .btn-close {
122     background-color: #6c757d;
123   }
124
125   .btn-close:hover {
126     background-color: #5a6268;
127   }
128
```

```

lab 14.4 > index5.html > html
2  <html lang="en">
3  <head>
7    <style>
125  .btn-close:hover {
126    background-color: #5a6268;
127  }
128
129  .btn-primary {
130    background-color: #28a745;
131  }
132
133  .btn-primary:hover {
134    background-color: #218838;
135  }
136  </style>
137 </head>
138 <body>
139  <h1>Notification Popup Modal Demo</h1>
140  <p>Click the buttons below to open the notification modal.</p>
141
142  <button onclick="openModal()">Open Announcement</button>
143  <button onclick="openAlertModal()">Open Alert</button>
144
145  <!-- Modal Overlay -->
146  <div class="modal-overlay" id="modalOverlay">
147    <!-- Modal Container -->
148    <div class="modal" id="modal">
149      <!-- Close Button (X) -->
150      <button class="modal-close-btn" onclick="closeModal()">&times;</button>
151
152      <!-- Modal Header -->
153      <div class="modal-header">
154        <h2 id="modalTitle">Announcement</h2>
155      </div>
156
157      <!-- Modal Body -->
158      <div class="modal-body" id="modalBody">
159        <p>This is an important notification for you. You can close this modal in multiple ways.</p>
160      </div>
161
162      <!-- Modal Footer -->
163      <div class="modal-footer">
164        <button class="btn-close" onclick="closeModal()">Close</button>
165        <button class="btn-primary" onclick="closeModal()">Got It</button>
166      </div>
167    </div>
168  </div>
169
170  <script>
171    const modalOverlay = document.getElementById('modalOverlay');
172    const modalTitle = document.getElementById('modalTitle');
173    const modalBody = document.getElementById('modalBody');
174
175    // Open Modal
176    function openModal() {
177      modalTitle.textContent = 'Announcement';
178      modalBody.innerHTML = '<p>This is an important announcement for all users. Please read carefully and take necessary action.</p>';
179      modalOverlay.classList.add('active');
180    }
181
182    // Open Alert Modal
183    function openAlertModal() {
184      modalTitle.textContent = 'Alert';
185      modalBody.innerHTML = '<p>⚠ This is an important alert! Please review the information and take appropriate action.</p>';
186      modalOverlay.classList.add('active');
187    }
188
189    // Close Modal
190    function closeModal() {
191      modalOverlay.classList.remove('active');
192    }
193
194    // Close Modal by clicking outside (on overlay)
195    modalOverlay.addEventListener('click', (event) => {
196      if (event.target === modalOverlay) {
197        closeModal();
198      }
199    });
200
201    // Close Modal on Escape key
202    document.addEventListener('keydown', (event) => {
203      if (event.key === 'Escape' && modalOverlay.classList.contains('active')) {
204        closeModal();
205      }
206    });
207  </script>
208 </body>
209 </html>

```

Output:-



Explanation:-

A modal popup is created using HTML, CSS, and JavaScript to display notifications. Clicking the “Open Modal” button shows the popup with a background overlay. The modal can be closed using a close button or by clicking outside the popup.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots